

Programming Assignment 3

Issue Date : 07.05.2026

Due Date : 22.05.2026 (23.59)

Programming Language : Oracle Java 8

Main Framework: JavaFX

T.A. : Res. Asst. İsmail Furkan Atasoy

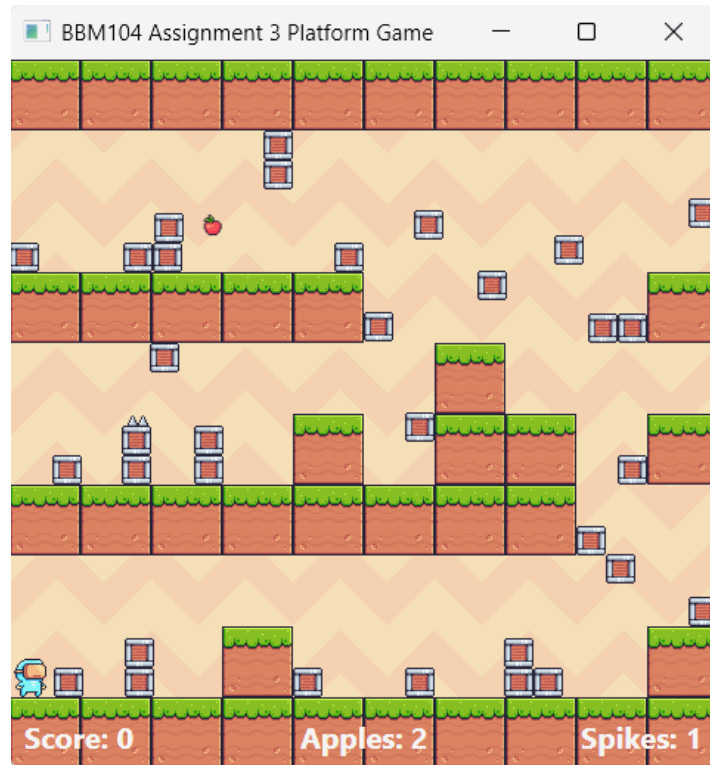


Figure 1: Welcome screen of the game.

1 Introduction

For this assignment, you are expected to develop a **2D platform game** using the **JavaFX framework** based on **Object-Oriented Programming (OOP) principles**. The project you will prepare is built around a **game character** moving on **platforms**, demonstrating jumping mechanics under the influence of gravity, and **interacting with some objects** in the game world (such as apples, spikes, etc.).

You are provided with **pre-made game assets** along with a text file named `level.txt` that contains their names and initial coordinates within the game area, as well as some **starter Java files**. You are expected to initialize the game using the given data and then **complete the necessary functionalities to make the game playable**.

1.1 Understanding the Game

In the game, **2 apples** and **1 spike** spawn randomly every **3 seconds** on the screen. The player's objective is to collect the apples by navigating the game world while **avoiding contact with spikes**.

When the player collects **2 apples**, they earn **1 score point**. For every **2 apples** collected, **1 random spike** is also removed from the map. Additionally, if the player has at least **5 score points**, they may spend **5 points** to destroy the nearest spike in the game area.

If the player collides with a spike, the player is reset to the **starting position** and all apples and spikes are removed from the game world, and a new game is automatically started.

The game contains multiple objects. For each object, pre-made assets are provided in the `assets` folder. For animated objects, frames are placed horizontally from left to right. These images follow the naming convention:

Object (WxHxF) .png

where:

- **W**: width in pixels
- **H**: height in pixels
- **F**: number of animation frames (1 if the object is static)

Player: A dynamic object (the only movable object in the game). It has an animation that runs continuously even in idle state. The player can move left, right, jump, and use a skill using keyboard inputs. It can collide with apples but cannot pass through terrain or blocks. Collision with a spike ends the game. Its asset name is `Player(22x26x11) .png`.

Apple: A static but animated object. It continuously plays its idle animation. Every 3 seconds, 2 apples randomly spawn in the air. When the player collides with an apple, it is considered collected and removed from the screen. Its asset name is `Apple(16x18x17) .png`.

Spike: A static object without animation. Every 3 seconds, 1 spike randomly spawns on top of a terrain or block. After every 2 apples collected, a random spike is removed from the game world, or a spike can be removed when the player uses a skill. Its asset name is `Spike(14x7x1) .png`.

Terrain: A static object without animation. These ground tiles are drawn at the start of the game and remain fixed throughout the game. The player can walk on them. Its asset name is `Terrain(48x48x1) .png`.

Block: A static object without animation. These are smaller platforms compared to terrain. They are drawn at the start of the game and remain fixed throughout the game. The player can also walk on them. Its asset name is `Block(20x20x1) .png`.

Background: The game background. It is automatically drawn at the start of the game from the provided starter code. Its asset name is `Background(960x480x1) .png`.

2 Task

Some core classes, namely `GameObject` and `GameEngine`, are provided as part of the starter code. However, you are required to implement the following classes yourself: `Player`, `Spike`, `Apple`, `Block`, and `Terrain`. Additionally, you must complete the missing parts of the methods in the `GameEngine` class.

2.1 `GameObject.java`

`GameObject` class is an **abstract base class** that represents all interactive and visual objects in the game world. This class provides common functionality such as **image loading**, **animation handling**, and **basic size management**, which are shared across all derived game objects (`Player`, `Apple`, `Spike`, `Block`, `Terrain`).

Note: This class is provided as a **pre-implemented component** and must not be modified under any circumstances. It is designed to serve as a foundational base class for all game objects, ensuring consistency in rendering, animation handling, and structural design across the project.

Constructor `GameObject(String path, double w, double h)`

The constructor of the `GameObject` class is responsible for initializing the **visual representation** of the object where:

- `path`: **File path** of the image asset.
- `w`: **Width** of a single animation frame.
- `h`: **Height** of a single animation frame.

The constructor loads the image from the given file path and adds the image view to the **JavaFX scene graph**. It is designed to be used by **subclasses** through inheritance by calling the constructor with appropriate parameters.

`animate(int totalFrames, int speed)` Method

In this method, the **animation system** is implemented using a **frame-based approach**. Its parameters are:

- `totalFrames`: Total number of frames in the **sprite sheet**.
- `speed`: Controls how often the frame changes (higher value = **slower animation**). You can **determine** an appropriate value **by experimenting**.

This method is provided as a **ready-to-use utility**. It must be called at appropriate points to ensure correct animation behavior for game objects that have animations, so that their **sprite animations** function properly during gameplay.

`getWidth()` and `getHeight()` Methods

They return the **width of the object** and the **height of the object** respectively. These ready-to-use methods will mainly be used for **spatial calculations** in the game engine.

update() Method

This method is intentionally left empty in the base class. It is designed to be overridden by **subclasses** specifically to handle **animation updates**.

2.2 <Object>.java

In this part, you are required to implement the core game object classes: **Player**, **Apple**, **Spike**, **Terrain**, and **Block**. Each of these classes must follow an object-oriented structure by extending the abstract **GameObject** class. This design ensures that all game entities inherit common rendering and animation capabilities while allowing specialized behavior where necessary.

All object classes in the game share a common structure. Each class:

- Extends the **GameObject** base class.
- Defines a constructor that initializes the object using its specific asset information.
- Passes appropriate parameters such as image path, frame width, frame height, number of animation frames, and animation speed to the superclass constructor.

These parameters are determined based on the provided asset specifications in the **assets** folder. The responsibility of each class is to correctly map its visual representation to the underlying game engine.

The **Spike**, **Terrain**, and **Block** classes represent static game elements. These objects do not require continuous animation updates during gameplay. Therefore, implementing the constructor method is sufficient and there is no need to override the **update()** method.

The **Player** and **Apple** classes are animated objects. They require continuous frame updates to create visual motion. Therefore, they must call the **animate()** method with appropriate parameters, inside their **update()** methods.

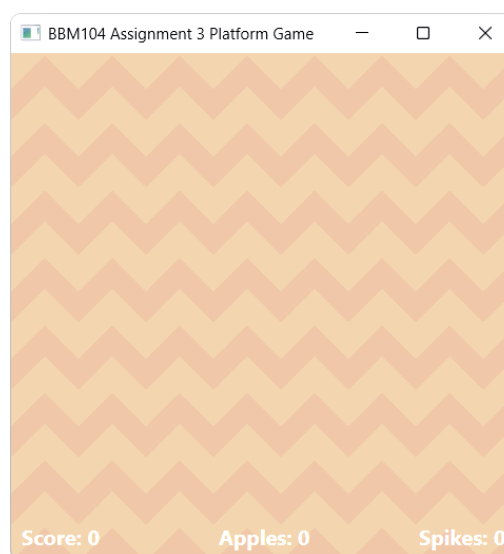


Figure 2: Empty Window After Implementing the Object Classes.

If you have implemented everything required so far, compiling and running the `GameEngine` should open a blank game window with the background already drawn (see Figure 2).

2.3 `GameEngine.java`

This is the class that contains **all the core logic of the game** and is required to run the game. You are expected to complete the missing parts in its methods to make the game playable.

Note: You are free to create new methods and attributes inside this class as you wish. However, remember that it must follow **Object-Oriented Programming (OOP)** principles.

Before you start implementing the methods, **it is recommended** to define separate lists for each type of game object (Terrain, Block etc.) as attributes. In the following sections, it will be assumed that you have defined these lists; however, you are free to proceed in a different way if you prefer. In addition, it will be assumed that you have defined the necessary attributes when needed.

`start(Stage stage)` Method

This method sets up the initial configuration of the game, registers event listeners, and starts the main game loop (**`GameLoop`**). Your task is to locate the **“TO DO”** comment line inside the relevant method in the provided starter file and implement the required code there.

For this method, you are expected to handle keyboard input and assign values to the corresponding attributes.

You must ensure that:

- Pressing the **“W”** key or the **“Up Arrow”** key triggers the **jump** action.
- Pressing the **“A”** key or the **“Left Arrow”** key moves the player to the **left**.
- Pressing the **“D”** key or the **“Right Arrow”** key moves the player to the **right**.
- Pressing the **"Space"** key triggers the **skill** action.

Within this method, it is sufficient to store which keys are currently being pressed and which keys have been released in the appropriate attributes.

For further details, you may refer to the "TO DO" comments provided inside the relevant method in the `GameEngine.java` file.

`reset()` Method

This method is called either at the beginning of the game or when the game ends. Your first task is to clear each object list defined in the `GameEngine` class back to its initial state.

Your second task is to read the object and coordinate data from the **`level.txt`** file and add these objects both to your own object lists and to the `gameRoot` object. In this way, your cleared and reset lists, as well as `gameRoot`, will be repopulated to their initial state.

For further details, you may refer to the "TO DO" comments provided inside the relevant method in the `GameEngine.java` file.

Note: When the tasks up to this point are completed, the initial objects should now be displayed on the screen, and the player animation should be visible. At this stage, the player is still floating in the air and there is no keyboard responsiveness implemented yet.



Figure 3: A Screenshot Taken After Implementing `reset()` Method.

isInvalid(GameObject obj) Method

This method is responsible for determining whether a given game object should be considered **invalid** and therefore the last action is rejected. An object is treated as invalid if it is either **out of the game boundaries** or **collides with any existing game object**.

First, the method checks whether the object is outside the visible game area defined by `GAME_WIDTH` and `GAME_HEIGHT`. If the object is within bounds, the next step is to check whether it intersects with any other game object.

The method should iterate through all relevant game object lists and checks for intersection. If any valid intersection is detected (excluding self-comparison), the object is considered invalid and the method must return `true`, otherwise `false`.

Note: To avoid false positives due to floating-point precision issues or border-touching cases, a small tolerance value (e.g., `0.1`) may optionally be added or subtracted from these comparisons.

For further details, you may refer to the "TO DO" comments provided inside the relevant method in the `GameEngine.java` file.

spawn() Method

This method is responsible for generating new **spikes** and **apples** at valid positions without violating game constraints.

You should follow the conditions below:

- Spawn apples at arbitrary positions in the air, ensuring that they do not intersect with any existing game object except spikes. Apples are allowed to overlap with each other.
- For spikes, randomly select a `Terrain` or `Block`. Place the spike directly on top of the selected platform at a random horizontal position, but ensure that it does not exceed the game boundaries. Spikes are allowed to overlap with other spikes and apples. If the selected position is otherwise invalid, select a different platform and retry. Do not spawn spikes on the topmost terrains in the game window where they would not be visible. Additionally, avoid spawning spikes between vertically stacked platforms where there is not enough space.

For further details, you may refer to the "TO DO" comments provided inside the relevant method in the `GameEngine.java` file.

update() Method

This method represents the **main game loop logic** and is automatically executed in every animation frame. It is responsible for updating all game objects, handling player movement, applying physics, detecting collisions, managing score updates, and controlling the spawning mechanism.

The method first iterates over all nodes in `gameRoot` and calls the `update()` method of each **GameObject**. This ensures that animated objects correctly update their animation frames.

Horizontal Movement (TO DO): Player movement along the x-axis is determined by keyboard input:

- If the left key is pressed, the player moves left by 1 pixel.
- If the right key is pressed, the player moves right by 1 pixel.

The player's direction must be visually updated where:

- 1 represents facing right,
- -1 represents facing left.

After updating the position, `isInvalid()` must be used to ensure the move is valid. If not, the movement should be reverted.

Vertical Movement and Gravity (TO DO): The vertical motion of the player is governed by gravity and jump mechanics:

- When the jump key is pressed and the player is on a surface, an initial upward velocity is applied, and a second jump is not allowed until the player lands on the ground.
- Gravity (0.1) continuously affects the player's vertical velocity.
- While the player is ascending, the velocity decreases until it reaches zero.
- After reaching the peak (approximately 55–60 pixels from the ground), the player begins to fall, accelerating downward.

You should use `isInvalid()` to perform collision checks, prevent the player from passing through platforms, and ensure proper landing behavior.

Spike Collision (TO DO): Check whether the player intersects with any Spike object. If a collision is detected, reset the game by calling `reset()` method.

Apple Collection and Scoring (TO DO): The player can collect apples through collision. When an apple is collected, it is removed from both the scene and the Apple object list. The **Apples** label should also be decremented by **1**. For every 2 apples collected, the score increases by **1**, and a random spike is removed from the game world (the **Spikes** label should be decremented by **1**).

Spawning (TO DO): Call the `spawn()` method every **3 seconds** to generate new objects. Each time a spike is spawned, the **Spikes** label should be incremented by **1**, and the **Apples** label should be incremented by **2**.

Spike Destroyer Skill (TO DO): If the player has **5 or more score points**, pressing the **Space** key destroys the **nearest spike** in the game world. The destroyed spike should be removed from the game area, the **Spikes** label should be decremented by **1**, and the **Score** label should be decremented by **5**. The **Apples** label should remain **unchanged**.

Camera Follow Player (TO DO): The camera system is responsible for keeping the player centered on the screen by horizontally shifting the entire game world.

- The player should remain **centered** in the screen during movement and the camera should follow the player along the **x-axis**. This can be achieved by translating the entire `gameRoot`.
- Camera scrolling should stop when the window boundaries align with the boundaries of the game world.

For further details, you may refer to the "TO DO" comments provided inside the relevant method in the `GameEngine.java` file.

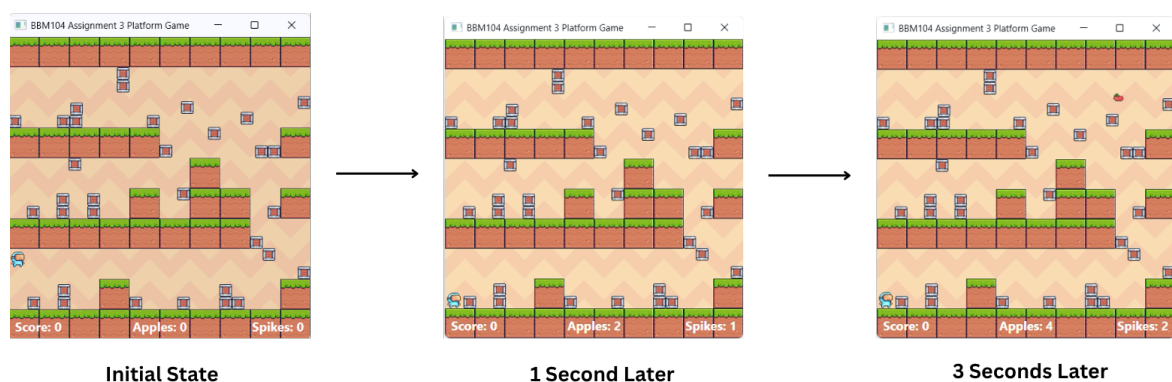


Figure 4: Example State 1.

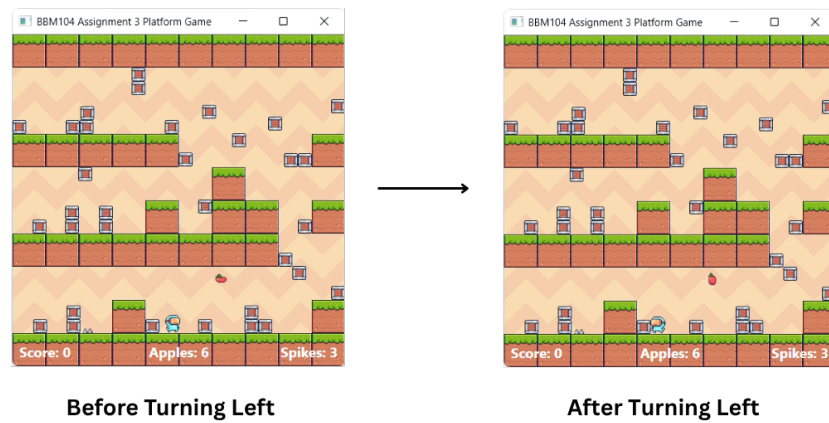


Figure 5: Example State 2.

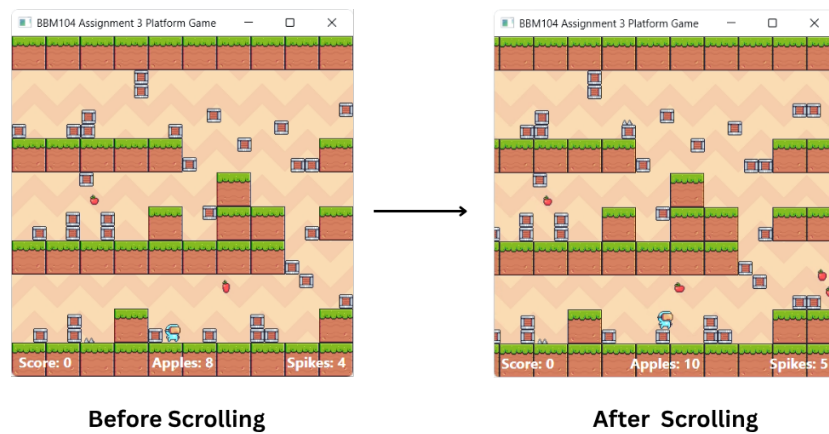


Figure 6: Example State 3.

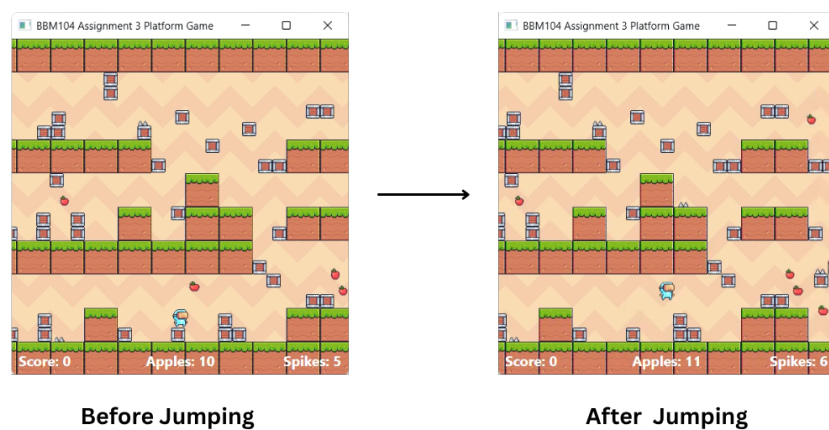


Figure 7: Example State 4.

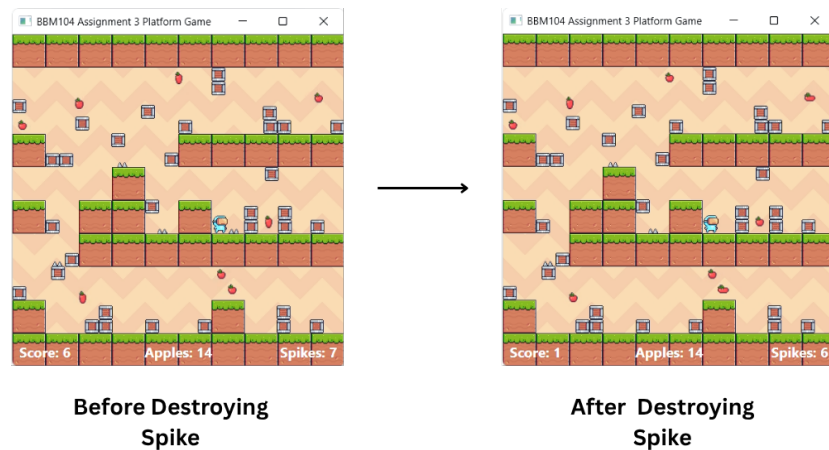


Figure 8: Example State 5.

3 Requirements

- The player must be able to move using **W**, **A** and **D** or **arrow keys** and use their skills with **Space** bar.
- The player and apple animations must run properly.
- The player must not move outside the **game boundaries** and must remain within the game area.
- The player must not pass through **Terrain** or **Block** objects; even if movement keys are pressed, the position must remain fixed upon collision.
- After jumping once, the player must not be able to jump again until landing on the ground.
- The player's horizontal movement speed must be exactly **1 pixel per frame**.
- The player's jumping mechanic must be realistic, using **gravity** and **velocity** calculations, where the player slows down while ascending and accelerates while descending. You must define appropriate **jump velocity** to ensure all parts of the map are reachable.
- The player's jump height should be approximately **55–60 pixels** from the **ground**, after which the player should start falling. This corresponds to slightly more than the height of a terrain tile.
- Every **3 seconds**, **2 apples** and **1 spike** must be spawned simultaneously. The **Apples** label should be incremented by **2**, and the **Spikes** label should be incremented by **1**.
- When an apple is collected, it must be removed from the game world and **Apple** label should be decremented by **1**.
- For every **2 apples** collected, a random spike must be removed from the game world. The **Spikes** label should be decremented by **1**, and the **Score** label should be incremented by **1**.

- If the player presses the **Space** key and has **5 or more score points**, the **nearest spike to the player** is destroyed. The destroyed spike is removed from the game area. The **Score** label is decremented by **5**, the **Spikes** label is decremented by **1**, and the **Apples** label remains unchanged.
- Apples that overlap with each other must be collected independently; collecting one apple must not be considered sufficient for all overlapping apples, and each apple must be checked and removed separately.
- Apples may spawn at any empty location in the game area, including mid-air. Apples are allowed to overlap with each other and spikes.
- Spikes must spawn on **platforms**, but they must not intersect with any objects other than apples or other spikes.
- Apples and especially spikes must spawn within the **game boundaries**.
- If the player collides with a spike, the game must reset and automatically restart from the initial state.
- Any error conditions must be properly handled, and the game must not crash under any circumstances.

4 Bonus Part (Optional)

After completing all requirements, you may create a new folder named `Bonus` and further develop your game in any way you want. The newly developed game must still be a 2D platform game and must include at least one controllable player character. You are free to modify assets, game mechanics, files and overall game engine design. However, the use of **JavaFX** is still required. As a reference and inspiration, you are encouraged to develop a game similar to the action game **Contra**.

If you decide to pursue bonus points and further improve your game, please ensure that it is still runnable through the `GameEngine` class inside the `Bonus` folder in the main directory. Your original core files must remain in the main directory without being removed.

5 Submitting

In the main directory, you must create a `README.md` file (if you are not familiar with how to edit an `.md` file, you may refer to external resources). In this file, you should include a publicly accessible YouTube demo video link. In the video, you should first present your source code, then compile and run the project, and finally demonstrate gameplay. You may either narrate the video or keep it silent. The video should clearly demonstrate as many of the required features as possible during gameplay.

If you have developed additional features under the Bonus section, you must provide two separate YouTube links: one for the base assignment implementation and one for your extended (bonus) version. In addition, you should explain your improved version of the game, including how it is played and its rules.

For students who do not implement the Bonus features, providing only the YouTube video link in the `README.md` file is sufficient (no need to explain the game mechanics, as they are already known).

File hierarchy must be compressed into a single **.zip** file because the system only supports **.zip** files. The zip file must be named using the following format: `"b<studentID>.zip"`.

Inside the archive, the project structure must exactly match the following hierarchy:

```
b<studentID>.zip
|--- assets
|--- Bonus/ (optional)
|--- Apple.java
|--- Block.java
|--- GameEngine.java
|--- GameObject.java
|--- level.txt
|--- Player.java
|--- README.md
|--- Spike.java
|--- Terrain.java
```

Note: The Bonus folder is optional. If you didn't improve the game, you may ignore it.

6 Compiling and Running

To **compile and run** your project, you can use the following commands in the terminal:

```
$ javac GameEngine.java
$ java GameEngine
```

7 Grading

There are no partial grading sections for this assignment. Your submission will be evaluated as a whole, and it will be graded out of **100** based on how well it satisfies the requirements. Afterwards, your score will be multiplied by a weighting factor between **0.9** and **1.0** based on the quality of your code (**OOP principles** and **clean code criteria**), and your final grade will be calculated accordingly. Finally, additional **bonus points** will be awarded to students who implement the **Bonus features**.